Usage guide

This covers everything in RSAS v3.2: the GUI, every settings dialog, the output formats, the scripting API, and the tips that save you time. If you just want to get a result on screen, the README's "60-second version" is faster. This is the reference you come back to when you need detail.

Starting the app

```
python main.py
```

The window opens on the Analyze page with a sidebar down the left. Seven pages:

Page	What it's for
Analyze	Load files, run the analysis, watch the log, export results
Results	The most recent run, shown across three read-only tabs (Results Table, Structure Preview, Full Log)
Settings	Analysis Settings, Performance, Output Columns, Motif Finder, Quality Score Builder, Sequence Options
Sequence Extractor	Pull upstream/downstream regions from local files or NCBI
Synthetic Pool	Generate random RNA pools with fixed motif inserts
RNArobo Search	Find structural motifs (helices and loops) with the bundled RNArobo engine
Pseudoknot Finder	Predict pseudoknotted structures with the bundled Knotty engine

Most of your time lives on Analyze and Settings. The other five pages are tools you reach for when a specific job comes up.

Loading input data

Formats it reads

- **FASTA** (`.fasta`, `.fa`): the usual, one or more sequences.
- **CSV / TSV** (`.csv`, `.tsv`): RSAS auto-detects the delimiter and works out which column is the name and which is the sequence from the header row, so you don't normally pick columns by hand. (From the scripting API you can override the columns explicitly; see the section below.)

- **Two-column text** (`.txt`): one `name<tab>sequence` per line.
- **GenBank** (`.gb` , `.gbk`): pulls the record sequences out. This one needs biopython installed.

A practical note: RSAS works in RNA. T's are converted to U automatically, and stray spaces, dashes, and pipe characters inside a sequence are stripped, but it's cleaner to start from tidy RNA so your output reads the way you expect.

How to load

Click **Browse** on the Analyze page and pick a file, or just drag the file onto the window. The path shows up in the input field, and you can type or paste a path there directly if that's easier. `Cmd+O` (macOS) or `Ctrl+O` (Windows/Linux) opens the file browser too.

Two things worth knowing. Drag-and-drop relies on an optional library (`tkdnd`); if it isn't installed, the drop silently does nothing and you just use Browse instead. And the "N sequences loaded" badge counts sequences that passed a basic nucleotide check, so if a few entries in your file are malformed they're skipped (with a warning in the log) and the count comes out lower than the number of records in the file.

New here? There's a full worked run, with real output numbers, in `Examples/README.md` .

Running an analysis

1. Load a file.
2. Adjust settings if you need to (sections below).
3. Click **Analyze**, or press `Cmd+R` / `Ctrl+R` .
4. Progress messages stream into the log as sequences get processed.
5. When it finishes, results are cached to `~/.rsas/Data/Outputs/rna_results.csv` and `rna_results.xlsx` . That's the working copy; Export (below) is how you save a named file wherever you want it.

How long it takes depends mostly on two things: how many sequences, and whether the partition function is on. A few dozen short leaders with MFE only is basically instant. A few thousand sequences with PF enabled is a coffee break, and that's where the CPU-cores setting earns its keep.

What happens to each sequence

For every sequence, RSAS:

1. Folds the full-length sequence at each temperature (MFE, plus PF if enabled).
2. Finds the hairpin, terminal or RBS-based depending on your setting.
3. Folds that hairpin on its own at each temperature.
4. Measures composition (AU%, GC%, GU%) for the full sequence and the hairpin.

5. Locates the Shine-Dalgarno / RBS and computes its paired percentage at each temperature.
6. Runs your motif search, if you turned it on, and measures sequestering for every hit.
7. Checks each MFE and composition value against the ranges you set.
8. Adds up the quality scores from how many checks passed.
9. Writes the enabled columns to CSV and Excel.

Features and tools

Every feature and tool has its own in-depth page under docs/features/: each one covers every option, what it produces, how it works, a worked example, and the gotchas. This is the quick map; follow a link for the detail.

On the Settings page:

- **Analysis Settings** configures the folding temperatures, the hairpin detection method (terminal vs RBS-based), and the RBS anchor and window.
- **Performance** sets the CPU core count for parallel runs (1 = sequential, which is fine for small runs; 2 to 4 helps on 100+ sequences). The dialog shows your machine's core count for reference.
- **Output Columns** picks which columns are computed and exported. Only enabled columns are computed, so trimming the ones you do not need makes runs faster. See the output column reference for what each column means.
- **Motif / Sequence Finder** searches every sequence for an IUPAC pattern.
- **Quality Score Builder** defines how candidates are scored and ranked.
- **Sequence Options** can append a sequence (an AUG, say) to the start or end of every input before analysis.

Sidebar tools:

- Sequence Extractor: pull upstream/downstream regions from local files or NCBI.
- Synthetic Pool Generator: build random RNA pools from a segment template.
- RNArobo Search: find structural motifs (helices and loops).
- Pseudoknot Finder: predict pseudoknots with Knotty.

Two features are configured indirectly: the partition function (ensemble analysis) is turned on by enabling its Output Columns, and composition is computed automatically. See their pages for detail.

Exporting results

When a run finishes:

1. Click **Export**, or press `Cmd+E` / `Ctrl+E`.

2. Pick a location. It defaults to Downloads with a timestamped filename so you don't clobber an earlier export.
3. Choose **Excel (.xlsx)** or **CSV (.csv)**.
4. Optionally open the containing folder afterward.

The Excel file

Up to three tabs:

Tab	What's in it
Full Sequence	Full-length data: structure per temperature, MFE, composition, RBS sequestering, range checks, quality scores
Hairpin Analysis	The extracted hairpin: detection method, sequence and structure, composition, MFE, PF data, quality scores
Motif Matches	One row per motif hit per sequence: position, matched subsequence, paired%, structure, PF accessibility per temperature, diffs

The Motif Matches tab only shows up if motif search was on and something matched. Each tab carries only the columns that were enabled when you ran the analysis.

The CSV file

One flat table with every enabled column. Where the motif finder produced multiple values for a sequence, those fields are semicolon-separated.

Scripting it without the GUI

You can run the analysis straight from Python.

```
from pathlib import Path
from RnaThermofinder.core import FastaParse, HairpinAnalysis

# Load sequences
sequences = FastaParse.read_fasta("input.fasta")

# For CSV/TSV input instead:
# sequences = FastaParse.read_csv_tsv_sequences(
#     "input.csv", skip_rows=30, name_col=0, seq_col=10
# )

# Analysis settings
settings = {
    "au_min": 50, "au_max": 60,
    "gc_min": 0, "gc_max": 30,
    "gu_min": 15, "gu_max": 25,
    "mfe_25_min": -17, "mfe_25_max": -10,
    "mfe_37_min": -13, "mfe_37_max": -6,
    "mfe_42_min": -7, "mfe_42_max": -2,
    "hairpin_detection_method": "terminal",  # or "rbs_based"
    "num_cpu_cores": 1,
    # Full-sequence ranges:
    "orig_mfe_25_min": -30, "orig_mfe_25_max": -10,
    "orig_mfe_37_min": -25, "orig_mfe_37_max": -5,
    "orig_mfe_42_min": -20, "orig_mfe_42_max": -2,
    "orig_au_min": 0, "orig_au_max": 100,
    "orig_gc_min": 0, "orig_gc_max": 100,
    "orig_gu_min": 0, "orig_gu_max": 100,
}

output_dir = Path("Data/Outputs")
output_dir.mkdir(parents=True, exist_ok=True)

results = HairpinAnalysis.calculate_results_final(
    sequences,
    output_dir,
    settings,
    progress_callback=print,
    csv_settings_manager=None,  # None = default columns
)
```

```
print(f"Analyzed {len(results)} sequences")
print(f"Results in {output_dir / 'rna_results.csv'}")
```

To control which columns get written, build a `SettingsManager` from `settings_manager.py` and pass it as `csv_settings_manager`. The same JSON file the GUI uses works here.

The motif finder on its own

```
from RnaThermofinder.utils.motif_finder import (
    find_motif_occurrences,
    analyze_motif_sequestering,
)

sequence = "AUGCGGAGGCUUAAGCUAGC"
motif = "GGAGG"

hits = find_motif_occurrences(sequence, motif, allow_overlap=True)
for h in hits:
    print(f"Match at {h['start']}-{h['end']}: {h['matched_seq']}")
```

The pool generator on its own

```
from RnaThermofinder.utils.synthetic_pool_generator import generate_pool, PRESETS

segments = PRESETS["RBS + AUG"]
result = generate_pool(
    n=10000,
    segments=segments,
    output_file="my_pool.fasta",
    seed=42,
)
print(f"Wrote {result['written']} sequences to {result['file']}")
```

Keyboard shortcuts

Shortcut	Action
`Cmd/Ctrl + O`	Open the file browser
`Cmd/Ctrl + R`	Run the analysis
`Cmd/Ctrl + E`	Export results

Tips from actually using it

- Start with the **Hairpin Analysis** preset for quick screening, then switch to **Full Export** once you've found something worth a closer look.
- Use **RBS-based detection** when the regulatory hairpin isn't the rightmost one, nested or multi-branch structures are exactly where terminal mode gets it wrong.
- **Turn off PF columns** when you don't need them. The partition function is the single most expensive part of a run, and most screening passes don't need ensemble numbers.
- The **temperature-difference columns are your main signal.** A thermometer shows a big drop in paired% at the RBS (or a motif) between the base temperature and the high one. That delta is what you're hunting for; everything else is context.
- **Match your temperatures to your biology.** Cold-shock element? Try 15/25/37. Heat-shock? 37/42/50. The defaults are a starting point, not a rule.
- **Quality scores come out sorted high to low,** so your best candidates are already at the top of the export.
- For big batches, bump the cores in **Performance** to 2 to 4. Past that you hit diminishing returns on most machines.
- In the **pool generator**, composition targets bias your library toward a GC range. Useful, but tight targets make rejection sampling slow, so loosen the tolerance if generation drags.

Where files live

Everything RSAS writes lives under `~/.rsas/` in your home folder, not next to the app. That's deliberate: it means the app keeps working when it's installed as a read-only bundle.

Path	What it is
`~/.rsas/Data/Outputs/rna_results.csv`	Working copy of the most recent run (CSV)
`~/.rsas/Data/Outputs/rna_results.xlsx`	Working copy of the most recent run (Excel)
`~/.rsas/csv_output_settings.json`	All your settings: columns, temperatures, RBS config, scoring profiles, presets (the name is historical; it holds more than columns)
`~/.rsas/.recent_files.json`	The recent-files list (created automatically)
`Examples/`	Sample FASTA files in the repo, to try things out

Export writes a separate, named copy wherever you choose (it defaults to your Downloads folder with a timestamped filename), so the `~/.rsas` copies are just the latest working output.

FAQ

Can I use DNA sequences with T instead of U? RSAS works in RNA (A, C, G, U). T's get folded as U, but convert them first so your output columns read the way you expect. Any sequence editor will do a find-and-replace.

How is the quality score calculated? Each score (hairpin and full-length) counts how many of your configured criteria fall in range. With the defaults that's 6 criteria, MFE at each of three temperatures plus three composition checks, so scores run 0 to 6. Change the criteria in the Quality Score Builder.

MFE versus PF, what's the difference? MFE gives you one structure, the single most stable one, where every base is either paired or not. The partition function considers the whole ensemble of possible structures weighted by energy, so instead of a yes/no it gives a probability of being unpaired at each position. PF is more informative and slower.

Can I add my own pool-generator presets? The only built-in is RBS + AUG, but you can build any layout by hand with the Add Segment buttons. If your lab reuses a layout, add it to the `PRESETS` dict in `synthetic_pool_generator.py`.

RNArobo or Pseudoknot Finder says the engine isn't available. Those tools call external programs that are bundled for macOS (`bin/macos/`). On Linux, download RNArobo's ready-made executable and build Knotty into `bin/linux/` of your RSAS folder (no administrator rights needed); on Windows, use WSL2. Commands: INSTALL.md, section 8.

How do I cite RSAS? See the Citation section in the README.